EMBEDDED LINUX НА ПРАКТИКЕ · КНИГА 3

Проверка плат

Что и как проверить после сборки и прошивки: загрузка, экран, ввод, звук, сеть, шины, датчики и питание — пошаговые процедуры для ODROID-GO Advance, BeagleBone Black, Raspberry Pi 1 и Raspberry Pi 4/400

SDK `github.com/sobs1992/linux-sdk`
Скрипт проверки: `/usr/bin/board-check.sh` в прошивке каждой платы
Октябрь 2026

Содержание

Предисловие **4**
Как устроена книга 4
Что нужно 4
Достоверность 4

1. Методика **5**
Перед записью карты 5
Запись 5
Журнал загрузки 5
Стадии загрузки 5
Автоматическая проверка 6
Протокол 6

2. Общие проверки Linux **7**
1. Ядро и командная строка 7
2. Корень и раздел BOOT 7
3. Модули 7
4. Журнал ядра 7
5. Консоли 7
6. Время 8
7. Сеть (где есть) 8
8. Перезагрузка и выключение 8
9. Длительный прогон 8

3. ODROID-GO Advance **9**
1. Загрузка — проверено на железе 9
2. Экран — проверено на железе 9
3. Кнопки — проверено на железе 9
4. Стик — проверено на железе 10
5. Демонстрация: oga-pad 10
6. Звук — ожидаемо 10
7. Питание — ожидаемо 10
8. Часы — ожидаемо 11
9. USB — ожидаемо 11
10. Разъём расширения P2 — ожидаемо 11
11. Датчики и частота — ожидаемо 11
Не работает по известным причинам 11

4. BeagleBone Black **12**
1. Загрузка с SD 12
2. EEPROM платы 12
3. Ethernet 12
4. Светодиоды USR0–USR3 13
5. HDMI 13
6. USB 13
7. Кнопка POWER 13
8. GPIO: P9_12 13
9. I2C2 на P9_19/P9_20 13
10. АЦП 14

11. Часы и частота 14
Не работает по известным причинам 14

5. Raspberry Pi 1 / Zero 15
1. Загрузка 15
2. Светодиоды 15
3. Ethernet (B, B+) 15
4. HDMI и клавиатура 15
5. USB 15
6. GPIO17 16
7. I2C1 (контакты 3 — SDA, 5 — SCL) 16
8. Датчики и частота 16
9. Звук 16
Не работает по известным причинам 16

6. Raspberry Pi 4 / 400 17
1. Загрузка 17
2. HDMI и клавиатура Pi 400 17
3. Ethernet 17
4. USB 17
5. Светодиоды 17
6. GPIO17 и I2C1 18
7. Датчики и частота 18
8. Звук 18
Не работает по известным причинам 18

7. Если проверка не прошла 19

A. Шаблон протокола испытаний 20

B. Скрипт board-check.sh 21

Предисловие

Образ собрался, записан на карту, плата загрузилась до `login:`. Значит ли это, что всё работает? Нет: экран может быть повёрнут не туда, звук молчать, а модуль Ethernet — не загрузиться. Эта книга — набор **процедур проверки**: что проверить на каждой плате, какой командой и что считать результатом.

Как устроена книга

- **Глава 1** — общая методика: подготовка, запись, журнал UART, стадии загрузки, автоматический скрипт `board-check.sh` и протокол испытаний.
- **Глава 2** — проверки Linux, одинаковые для всех плат.
- **Главы 3–6** — процедуры для каждой платы: OGA, BBB, RPi 1, RPi 4/400.
- **Глава 7** — что делать, если проверка не прошла.
- **Приложения** — шаблон протокола и листинг скрипта.

Что нужно

Что	Зачем
USB-UART адаптер на 3,3 В	журнал загрузки, консоль
Картридер и запасная SD-карта	прошивка, восстановление
Сетевой кабель, роутер с DHCP	BBB, RPi
USB-флешка	проверка USB-хоста
Наушники / динамик, HDMI-монитор	звук, видео
Светодиод + резистор 330–1000 Ом, перемычки	GPIO, UART-петля
USB-клавиатура (для RPi 4, BBB с HDMI)	консоль на экране

Достоверность

Каждая проверка помечена:

- **проверено на железе** — так было на настоящей плате при создании SDK;
- **ожидаемо** — результат выведен из исходников, конфигурации ядра и DTS, но на железе ещё не наблюдался.

На момент написания на железе проверялась только ODROID-GO Advance Black Edition: загрузка, экран с поворотом, кнопки и стик. Всё остальное — ожидаемые результаты, и протокол испытаний (приложение А) нужен, чтобы превратить их в проверенные.

> **КАК ЧИТАТЬ РЕЗУЛЬТАТЫ**
> Если результат проверки отличается от описанного, это не обязательно ошибка платы: имена устройств, номера и порядок могут меняться между версиями ядра. Ошибка — когда **функция** не работает (нет звука, кнопка не даёт событий).

ГЛАВА 1

Методика

Перед записью карты

```
make info                       # строка Firmware: — та ли плата выбрана?
make FW=<fw> image              # образ собран из текущих компонентов
sfdisk -l output/<fw>/images/<fw>.img
mtype -i output/<fw>/image-work/boot.vfat ::/extlinux/extlinux.conf
```

Проверьте в `extlinux.conf` командную строку ядра: консоль и параметры, которые вы меняли (например, `fbcon=rotate:3` для OGA). Это дешевле, чем выяснять на плате.

Запись

```
lsblk                                   # какое устройство — карта
make FW=<fw> flash DEV=/dev/sdX         # весь образ
```

`make flash` пишет образ **выбранной** прошивки; `FW=` задаёт её явно.

Журнал загрузки

Подключите UART (глава 2 второй книги) и запишите загрузку в файл **с момента включения** — потом его можно сравнить с ожидаемым:

```
stty -F /dev/ttyUSB0 115200 raw -echo cs8 -cstopb -parenb -crtscts
timeout 120 cat /dev/ttyUSB0 | tee boot-<fw>-$(date +%F).log
```

Или `picocom -b 115200 --logfile boot.log /dev/ttyUSB0`.

> **ОДИН ЧИТАТЕЛЬ НА ПОРТ**
> Пока `cat` или `screen` держат порт, второй процесс получит часть данных или ошибку «Device or resource busy». После записи журнала закройте захват.

Стадии загрузки

Первая проверка на любой плате — до какой стадии дошла загрузка:

Стадия	Признак в журнале UART	Если не дошли
Загрузчик ROM/прошивки	у OGA — короткий мусор (1,5 Мбод), у RPi — ничего, у BBB — `CCCC` при отсутствии загрузчика	плата не видит карту/загрузчик
U-Boot	`U-Boot 2026.07 (…)`	книга 1, глава 11
Выбор dtb	`Retrieving file: /…dtb`	книга 2, глава 9
Ядро	`Starting kernel …`, затем `Linux version 7.2.9-<fw>`	`earlycon` в командной строке
Корень	`VFS: Mounted root (ext4 filesystem)`	`root=PARTUUID`, драйвер SD
init	`Run /sbin/init as init process`	rootfs
Вход	`<hostname> login:`	getty, консоль

Автоматическая проверка

Скрипт `board-check.sh` за несколько секунд проходит по всем подсистемам и печатает строки `OK`, `WARN`, `FAIL`, `INFO`. Он рассчитан на busybox и не требует пакетов.

Скрипт входит в прошивку всех плат: он лежит в общем overlay `board/common/buildroot/rootfs-overlay/usr/bin/board-check.sh` и после `make rootfs` оказывается в `/usr/bin/`. Если на карте старый образ без него, скопируйте вручную:

```
scp board/common/buildroot/rootfs-overlay/usr/bin/board-check.sh root@board:/usr/bin/
```

Изменили скрипт — пересоберите rootfs и образ (`make rootfs image`): overlay копируется заново при каждой сборке rootfs.

Запуск на плате (от root):

```
# board-check.sh | tee /root/check-$(date +%F).txt
```

Код возврата — число строк `FAIL`. Строки `WARN` требуют внимания, но могут быть нормой: например, «нет /dev/fb0» у BBB без монитора.

> **КАК ПРОВЕРЕН СКРИПТ**
> Синтаксис — `dash -n` и `busybox sh -n`. Запуск — на ПК в окружении из одних апплетов busybox (строки про `/boot` и запись в `/root` там ожидаемо `FAIL`). На платах скрипт ещё не запускался.

Протокол

Каждую проверку записывайте: дата, плата и её ревизия, коммит SDK, результат, примечание. Шаблон — в приложении А. Без протокола через месяц невозможно сказать, работал ли звук на RPi 1 и с каким ядром.

```
git -C ~/linux-sdk describe --always --dirty     # версия SDK для протокола
```

ГЛАВА 2

Общие проверки Linux

Эти проверки одинаковы для всех плат. Большую часть делает `board-check.sh`; здесь — что за ними стоит и что проверить руками.

1. Ядро и командная строка

```
# uname -r                          7.2.9-<fw>: oga, bbb, rpi1 или rpi4
# cat /proc/device-tree/model       модель из dtb
# cat /proc/cmdline                 root=PARTUUID=…-02, консоли, ваши параметры
```

Норма: версия заканчивается на имя прошивки, модель — ваша плата и ревизия, командная строка совпадает с `LINUX_CMDLINE` конфига прошивки.

2. Корень и раздел BOOT

```
# mount | grep -E ' / | /boot '
/dev/root on / type ext4 (rw,…)
/dev/mmcblk0p1 on /boot type vfat (rw,noatime,…)
# ls /boot/extlinux/extlinux.conf
# echo test > /root/t && sync && cat /root/t && rm /root/t
```

Норма: корень `ext4` на запись, `/boot` смонтирован (по `LABEL=BOOT`, имя устройства у RPi 4 может быть `mmcblk1p1`), запись проходит.

3. Модули

```
# ls /lib/modules/$(uname -r)
# lsmod | head
# dmesg | grep -i 'disagrees about version\|Invalid module format'
```

Норма: каталог модулей для текущего ядра есть, ошибок формата нет. Если каталога нет — в rootfs модули другого ядра: пересоберите и запишите образ целиком (`make linux image`, `make flash`).

4. Журнал ядра

```
# dmesg | grep -n -E 'Oops|BUG:|Call trace|Kernel panic'     должно быть пусто
# dmesg | grep -i -E 'error|fail' | head -30                 изучить каждую строку
```

Строки с `error` не всегда ошибки: драйвер может пробовать несколько вариантов. Сравнивайте с журналом предыдущей рабочей сборки.

5. Консоли

- **UART**: приглашение `login:`, вход `root` без пароля, `Ctrl-C` и `Ctrl-D` работают, длинная строка переносится.

- **Экран** (`tty1`): лог загрузки и приглашение на дисплее.

```
# ps | grep getty                  getty на UART и на tty1
```

6. Время

```
# date
# hwclock -r                       есть, если на плате RTC (OGA, BBB)
```

Без сети и без RTC с батарейкой часы стартуют с 1970 года — это не ошибка.

7. Сеть (где есть)

```
# ip -4 addr                       адрес от DHCP на eth0
# ping -c 3 <адрес ПК>
```

С ПК: `ssh root@<адрес>` (если задан пароль или ключ, вторая книга, глава 2).

8. Перезагрузка и выключение

```
# reboot                           плата должна загрузиться снова
# poweroff                         в журнале UART: "reboot: Power down"
```

Проверьте несколько перезагрузок подряд: проблемы инициализации (SD, DDR) бывают нестабильными.

9. Длительный прогон

Короткая проверка не находит перегрев, утечки и нестабильность питания. Простой прогон на 30–60 минут:

```
# for i in 1 2 3 4; do (while :; do :; done) & done     нагрузить все ядра
# while :; do cat /sys/class/thermal/thermal_zone0/temp; sleep 10; done
# dd if=/dev/zero of=/root/big bs=1M count=100 conv=fsync && md5sum /root/big && rm /root/big
# kill %1 %2 %3 %4
```

Норма: плата не перезагружается, температура стабилизируется, `dmesg` без новых ошибок, записанный файл читается.

ГЛАВА 3

ODROID-GO Advance

Плата: RK3326, 4 × Cortex-A35, 1 ГБ. Консоль — `ttyS2`, 115200. Прошивка `oga`.

1. Загрузка — проверено на железе

Журнал UART Black Edition с заводским загрузчиком во SPI:

```
<короткий мусор: DDR-блоб и miniloader на 1,5 Мбод>
U-Boot 2026.07 (…)
Model: ODROID-GO Advance
DRAM:  1 GiB (total 1022 MiB)
** Booting bootflow 'mmc@ff370000.bootdev.part_1' with extlinux
Retrieving file: /Image
Retrieving file: /rockchip/rk3326-odroid-go2-v11.dtb
Starting kernel ...
[    0.000000] Machine model: ODROID-GO Advance Black Edition
…
odroid-go login:
```

Проверка	Норма
Выбор dtb	BE → `rk3326-odroid-go2-v11.dtb`; rev 1.0 → `rk3326-odroid-go2.dtb`; Super → `rk3326-odroid-go3.dtb`
Синий светодиод	загорается в U-Boot, в Linux мигает (`heartbeat`)
Безобидные строки	`Loading Boot0000 'mmc 0' failed`, `mmc0: error -110 whilst initialising SD card` (вторая попытка успешна), `deferred probe pending … wifi-pwrseq`

2. Экран — проверено на железе

```
# cat /sys/class/graphics/fbcon/rotate          3
# cat /sys/class/graphics/fb0/virtual_size
# ls /sys/class/backlight/
# cd /sys/class/backlight/* && cat max_brightness
# echo 10 > brightness ; echo $(cat max_brightness) > brightness
```

Норма: лог и приглашение на экране, текст читается в альбомной ориентации (`fbcon=rotate:3`), яркость меняется.

> **ОРИЕНТАЦИЯ**
> Правильная ориентация на Black Edition — `rotate:3`; подсказка из DTS (`rotation = <270>`) дала бы перевёрнутую картинку. Без `CONFIG_FRAMEBUFFER_CONSOLE_ROTATION` параметр игнорируется (он включён во фрагменте OGA).

3. Кнопки — проверено на железе

```
# evtest                    выбрать gpio-keys
```

Нажмите **каждую** кнопку и отметьте в протоколе событие `EV_KEY … value 1` (нажатие) и `value 0` (отпускание):

Кнопки	Коды
Крестовина	`BTN_DPAD_UP`, `_DOWN`, `_LEFT`, `_RIGHT`
A, B, X, Y	`BTN_EAST`, `BTN_SOUTH`, `BTN_WEST`, `BTN_NORTH` (соответствие надписям проверьте по событиям)
F1–F6	`BTN_TRIGGER_HAPPY1` … `BTN_TRIGGER_HAPPY6`
Курки	`BTN_TL`, `BTN_TR`
Вторые курки (только Black Edition)	`BTN_TL2`, `BTN_TR2`

Итого 16 кнопок у rev 1.0, 18 у Black Edition. Кнопка питания — отдельное устройство `rk805 pwrkey` (`KEY_POWER`).

4. Стик — проверено на железе

```
# evtest                    выбрать adc-joystick
```

Норма: при отклонении меняются `ABS_X` и `ABS_Y`, в покое значения возвращаются к центру, крайние положения дают значения у границ диапазона (диапазон evtest печатает в начале: `Min`, `Max`). Запишите в протокол центр и крайние значения: при износе стика центр «уплывает».

5. Демонстрация: oga-pad

```
# oga-pad
```

Квадрат в центре экрана, крестовина двигает его в ту же сторону, куда нажата, A/B/X/Y меняют цвет, F6 — выход. Проверяет сразу ввод, framebuffer и поворот.

6. Звук — ожидаемо

```
# aplay -l                  карта на кодеке RK817
# amixer                    регуляторы громкости
# speaker-test -t sine -f 440 -l 1
```

Проверьте динамик и наушники отдельно. Вставка наушников — событие на входном устройстве `rk817_int Headphones` (`evtest`, `SW_HEADPHONE_INSERT`). Если тишина — ищите в `amixer contents` переключатель выхода и громкость, отключённую по умолчанию.

7. Питание — ожидаемо

```
# ls /sys/class/power_supply/
# cat /sys/class/power_supply/*/status /sys/class/power_supply/*/capacity
```

Норма: при подключённом питании USB-C статус `Charging`, без него `Discharging`, ёмкость в процентах меняется со временем. Драйвер зарядки RK817 — модуль (`lsmod | grep rk817`). Красный светодиод `red:charging` горит при зарядке.

8. Часы — ожидаемо

```
# hwclock -r                  RTC в PMIC RK817 (rk808-rtc)
# date -s "2026-10-11 12:00" && hwclock -w
```

Выключите питание на минуту, включите: `hwclock -r` должен показать время, ушедшее вперёд на эту минуту (пока батарея не разряжена).

9. USB — ожидаемо

Вставьте флешку в USB-разъём:

```
# dmesg | tail                sd 0:0:0:0: [sda] … Attached SCSI removable disk
# mount /dev/sda1 /mnt && ls /mnt && umount /mnt
```

Сеть через USB-Ethernet адаптер: появится `eth0`, `udhcpc -i eth0` получит адрес.

10. Разъём расширения P2 — ожидаемо

UART1, петля. Соедините перемычкой контакт 2 (RXD) и 3 (TXD):

```
# stty -F /dev/ttyS1 115200 raw -echo
# cat /dev/ttyS1 &
# echo hello > /dev/ttyS1            должно напечататься hello
# kill %1
```

I2C1 (контакты 7 — SCL, 8 — SDA). Нужен пакет `i2c-tools`:

```
# i2cdetect -y 1                     без устройств — все "--"
```

С подключённым датчиком появится его адрес.

11. Датчики и частота — ожидаемо

```
# cat /sys/class/thermal/thermal_zone*/type /sys/class/thermal/thermal_zone*/temp
# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

Под нагрузкой (глава 2, прогон) частота растёт, температура не уходит за 80 °C.

Не работает по известным причинам

Что	Почему
Wi-Fi Black Edition (ESP8089)	драйвера нет в mainline
USB-гаджет через OTG	не проверялся; разъём может быть разведён только как хост

ГЛАВА 4

BeagleBone Black

Плата: AM335x, 1 × Cortex-A8, 512 МБ, eMMC 4 ГБ. Консоль — `ttyS0`, разъём J1 (1 — GND, 4 — RX, 5 — TX), 115200. Прошивка `bbb`. Все результаты — **ожидаемые**.

1. Загрузка с SD

BBB по умолчанию грузится с eMMC. Чтобы загрузиться с карты: снять питание, зажать **S2** (кнопка у слота SD), подать питание, отпустить через 2–3 секунды.

```
U-Boot SPL 2026.07 (…)
Trying to boot from MMC1
U-Boot 2026.07 (…)
Model: TI AM335x BeagleBone Black
…
Retrieving file: /zImage
Retrieving file: /am335x-boneblack.dtb
Starting kernel ...
Linux version 7.2.9-bbb …
beaglebone login:
```

Проверка	Норма
Загрузились с SD, а не с eMMC	дата сборки в строке `U-Boot 2026.07 (…)` совпадает с вашей сборкой; ядро `7.2.9-bbb`
dtb	`am335x-boneblack.dtb` (у Green — `am335x-bonegreen.dtb`)
eMMC видна	`ls /dev/mmcblk1*` — **ничего туда не пишите**

Если в журнале только символы `CCCC…` — ROM не нашёл загрузчик ни на eMMC, ни на SD и ждёт загрузки по UART: проверьте карту (`MLO` в корне активного FAT-раздела).

2. EEPROM платы

```
# hexdump -C /sys/bus/i2c/devices/0-0050/eeprom | head -2
```

Норма: в первых байтах читается `A335BNLT` и ревизия платы. Драйвер `at24` — модуль, файл появляется после его загрузки (mdev делает это сам).

3. Ethernet

```
# ip -4 addr show eth0        адрес от DHCP
# ping -c 3 <адрес ПК>
```

Светодиоды на разъёме RJ45 горят. С ПК — `ssh root@<адрес>` (после настройки пароля или ключа).

4. Светодиоды USR0–USR3

```
# ls /sys/class/leds/
beaglebone:green:heartbeat  beaglebone:green:mmc0  beaglebone:green:usr2
beaglebone:green:usr3
# cd /sys/class/leds/beaglebone:green:usr3
# echo none > trigger; echo 1 > brightness; echo 0 > brightness
```

Норма: USR0 мигает «сердцебиением», USR1 — при обращениях к SD, USR2/USR3 включаются вручную.

5. HDMI

Подключите монитор **до** включения.

```
# lsmod | grep -E 'tilcdc|tda998x'
# ls /dev/fb0
```

Норма: на мониторе лог и приглашение `tty1`, USB-клавиатура вводит текст.

6. USB

Флешка в разъём USB-A: `dmesg | tail`, `mount /dev/sda1 /mnt`.

7. Кнопка POWER

```
# evtest                        устройство кнопки от PMIC TPS65217
```

Короткое нажатие — событие `KEY_POWER`.

8. GPIO: P9_12

Светодиод с резистором между P9_12 и GND (P9_1). Номер контроллера банка 1 — по адресу `4804c000` (`board-check.sh` печатает соответствие, или `gpiodetect`):

```
# gpio-blink /dev/gpiochipN 28 10 500
```

Норма: светодиод мигает 10 раз. Если программа отработала, а светодиод не мигает — вывод не переведён в режим GPIO: примените overlay `bbb-sdk-led.dtso` с pinmux (вторая книга, глава 8).

9. I2C2 на P9_19/P9_20

```
# i2cdetect -y -r 2             пакет i2c-tools
```

Адреса `0x54`–`0x57` описаны в DTS как EEPROM плат расширения: при наличии драйвера `at24` они показываются как `UU`.

10. АЦП

```
# cat /sys/bus/iio/devices/iio:device0/in_voltage0_raw      AIN0 = P9_39
```

ТОЛЬКО 1,8 В

Входы АЦП AM335x рассчитаны на 0–1,8 В. Подача 3,3 В повредит процессор. Опорное напряжение — VDD_ADC (P9_32), земля — GNDA_ADC (P9_34).

11. Часы и частота

```
# hwclock -r                    RTC AM335x (rtc-omap); батарейки нет — время не сохраняется
без питания
# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

Не работает по известным причинам

Что	Почему
Wi-Fi/BT у BeagleBone Black Wireless	нет прошивки чипа WL18xx в rootfs
PRU	не настроены в SDK
Загрузка без кнопки S2	ROM выбирает eMMC, если там есть загрузчик

ГЛАВА 5

Raspberry Pi 1 / Zero

Плата: BCM2835, 1 × ARM1176 (ARMv6), 256–512 МБ. Консоль — `ttyAMA0`, гребёнка (6 — GND, 8 — TXD, 10 — RXD), 115200. Прошивка `rpi1`. Все результаты — **ожидаемые**.

1. Загрузка

Прошивка GPU ничего не печатает в UART. На HDMI на мгновение появляется цветной экран прошивки, затем:

```
U-Boot 2026.07 (…)
Model: Raspberry Pi Model B+
…
Retrieving file: /zImage
Retrieving file: /bcm2835-rpi-b-plus.dtb
Starting kernel ...
Linux version 7.2.9-rpi1 …
raspberrypi login:
```

Проверка	Норма
dtb по модели	B+ → `bcm2835-rpi-b-plus.dtb`, B rev2 → `bcm2835-rpi-b-rev2.dtb`, Zero → `bcm2835-rpi-zero.dtb`; `printenv fdtfile` в U-Boot
U-Boot стартовал	если в UART тишина, а на HDMI цветной экран остался — прошивка не нашла dtb под старым именем `bcm2708-rpi-*.dtb` (первая книга, глава 13)

2. Светодиоды

```
# ls /sys/class/leds/            ACT, PWR
```

ACT мигает «сердцебиением» (триггер `heartbeat`).

3. Ethernet (B, B+)

Ethernet висит на USB-хабе (LAN9512/9514):

```
# dmesg | grep smsc95xx
# ip -4 addr show eth0
```

У A/A+ и Zero Ethernet нет.

4. HDMI и клавиатура

Лог и приглашение на мониторе (`tty1`), USB-клавиатура работает. Драйвер VC4 встроен.

5. USB

Флешка: `dmesg | tail`, `mount /dev/sda1 /mnt`. У B/B+ порты и Ethernet делят один USB 2.0: при копировании по сети скорость ограничена.

6. GPIO17

Светодиод с резистором между контактом 11 (GPIO17) и 9 (GND):

```
# gpio-blink /dev/gpiochip0 17 10 500
```

7. I2C1 (контакты 3 — SDA, 5 — SCL)

```
# i2cdetect -y 1                пакет i2c-tools
```

8. Датчики и частота

```
# cat /sys/class/thermal/thermal_zone0/temp       BCM2835 thermal
# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
```

9. Звук

```
# aplay -l
```

Ожидается звуковая карта HDMI от VC4. Аналоговый выход 3,5 мм в mainline идёт через драйвер staging `snd-bcm2835` (модулем) и может не появиться.

Не работает по известным причинам

Что	Почему
Wi-Fi Zero W	нет прошивки чипа в rootfs
`/proc/config.gz`	`IKCONFIG_PROC` выключен в `bcm2835_defconfig`
Overlay во время работы	`OF_OVERLAY` выключен (U-Boot overlay применяет)

ГЛАВА 6

Raspberry Pi 4 / 400

Плата: BCM2711, 4 × Cortex-A72, 2–8 ГБ (Pi 400 — 4 ГБ). Консоль на гребёнке — mini-UART `ttyS1` (контакты 6, 8, 10), 115200. Pi 400: основная консоль — HDMI и встроенная клавиатура. Прошивка `rpi4`. Все результаты — **ожидаемые**.

1. Загрузка

```
U-Boot 2026.07 (…)
Model: Raspberry Pi 400 Rev 1.x            (или Raspberry Pi 4 Model B …)
…
Retrieving file: /Image
Retrieving file: /broadcom/bcm2711-rpi-400.dtb
Starting kernel ...
Linux version 7.2.9-rpi4 …
raspberrypi login:
```

Проверка	Норма
dtb	`broadcom/bcm2711-rpi-400.dtb` или `-rpi-4-b.dtb`; прошивке нужен тот же файл в корне BOOT
UART молчит	`enable_uart=1` в `config.txt`, адаптер на контактах 8/10
Ничего на экране и в UART	EEPROM показывает диагностический экран по HDMI — прочитайте сообщение

2. HDMI и клавиатура Pi 400

Монитор — в порт с подписью **HDMI0** (прошивка и консоль выводятся на него). На экране — лог и приглашение, встроенная клавиатура Pi 400 вводит текст (контроллер USB VL805 на PCIe, драйверы встроены). Для Pi 4 B — любая USB-клавиатура.

```
# dmesg | grep -i -E 'xhci|vl805|pcie'
```

3. Ethernet

```
# dmesg | grep -i genet
# ip -4 addr show eth0
```

4. USB

Флешка в синий порт USB 3.0, затем в чёрный USB 2.0: оба должны определяться.

```
# dmesg | tail                 new SuperSpeed USB device … (для USB 3.0)
```

5. Светодиоды

```
# ls /sys/class/leds/         Pi 4 B: ACT, PWR; Pi 400: только PWR
```

6. GPIO17 и I2C1

```
# gpio-blink /dev/gpiochip0 17 10 500
# i2cdetect -y 1               контакты 3 и 5; драйвер i2c-bcm2835 — модуль
```

7. Датчики и частота

```
# cat /sys/class/thermal/thermal_zone0/temp
# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

Драйверы температуры и частоты (`bcm2711_thermal`, `raspberrypi-cpufreq`) — модули: если файлов нет, проверьте `lsmod`. Под нагрузкой без радиатора Pi 4 нагревается до 80 °C и снижает частоту.

8. Звук

```
# aplay -l                     HDMI-звук через VC4
# speaker-test -D hw:0 -c 2 -t sine
```

Не работает по известным причинам

Что	Почему	Как включить
Wi-Fi, Bluetooth	драйвер `brcmfmac` есть, но нет файлов прошивки чипа: в `dmesg` будет ошибка загрузки прошивки `brcmfmac`	`BR2_PACKAGE_BRCMFMAC_SDIO_FIRMWARE_RPI=y`, `_WIFI=y`, `_BT=y` в defconfig платы
ACT на Pi 400	на плате его нет	—

ГЛАВА 7

Если проверка не прошла

Симптом	Что проверить	Где подробнее
Ничего в UART, плата «мёртвая»	скорость 115200, TX↔RX, общая земля, питание; у BBB — кнопка S2; у OGA — загрузчик во SPI	книга 1, гл. 11
Мусор в UART	скорость; кто печатает (загрузчик ROM/SPI на 1,5 Мбод)	книга 1, гл. 11
U-Boot есть, ядра нет	`bootflow scan -l`, `printenv fdtfile`, файлы на BOOT, `fdtoverlay_addr_r` при overlay	книга 2, гл. 9
Ядро молчит после `Starting kernel`	`earlycon` в командной строке, адреса загрузки	книга 2, гл. 8
`VFS: Unable to mount root fs`	`root=PARTUUID`, драйвер SD встроен, раздел записан	книга 1, гл. 10
Нет приглашения	getty на нужной консоли, `console=`	книга 1, гл. 8
Модули не грузятся	версия ядра и `/lib/modules`, полная перепрошивка	книга 2, гл. 7
Параметр ядра не действует	включена ли опция Kconfig, которая его читает (пример — `fbcon=rotate`)	книга 1, гл. 14
Устройство не появилось	`devices_deferred`, узел в DTS, драйвер в `.config`	книга 2, гл. 8
Нестабильная работа	питание (блок не меньше 2 А, для Pi 4 — 3 А), нагрев, карта памяти	глава 2

СНАЧАЛА — ЧТО ИЗМЕНИЛОСЬ

Самый быстрый путь к причине — сравнить с последней рабочей сборкой: журнал UART, `dmesg`, `board-check.sh`, `git log` в SDK и сабмодулях. Поэтому протокол испытаний стоит вести с первого дня.

ПРИЛОЖЕНИЕ А

Шаблон протокола испытаний

Плата		Ревизия		Прошивка	
Коммит SDK		Дата		Проверял	
SD-карта		Блок питания		Адаптер UART	

№	Проверка	Ожидание	Результат	OK	Примечание
1	Загрузка до `login:`	все стадии по журналу UART			
2	`board-check.sh`	0 FAIL			
3	Корень на запись, `/boot`	rw, LABEL=BOOT			
4	Модули	каталог для `uname -r`			
5	Консоль UART	вход root			
6	Экран	лог и приглашение, ориентация			
7	Ввод	все кнопки/клавиатура			
8	Звук	динамик/наушники/HDMI			
9	Сеть	DHCP, ping, ssh			
10	USB	флешка монтируется			
11	GPIO	светодиод мигает			
12	I2C	`i2cdetect` видит устройство			
13	Часы	`hwclock -r`			
14	Датчики, частота	температура, cpufreq			
15	Питание	заряд/статус (OGA)			
16	Перезагрузка ×3, выключение	без ошибок			
17	Прогон 30–60 мин	без сбоев			

Журнал UART: `boot-<fw>-<дата>.log` · отчёт скрипта: `check-<дата>.txt`

ПРИЛОЖЕНИЕ B

Скрипт board-check.sh

Скрипт лежит в общем overlay rootfs и попадает в прошивку всех плат как `/usr/bin/board-check.sh`.

board/common/buildroot/rootfs-overlay/usr/bin/board-check.sh

```
#!/bin/sh
# board-check.sh: automatic report on a running board (OdroidLinux SDK).
#
# Runs with busybox sh, needs no extra packages. Prints one line per check:
#   OK    works as expected
#   WARN  worth a look (may be normal for this board)
#   FAIL  broken
#   INFO  facts to write into the test report
# Exit code: number of FAIL lines.

fails=0
ok()      { printf 'OK    %s\n' "$*"; }
warn()    { printf 'WARN  %s\n' "$*"; }
fail()    { printf 'FAIL  %s\n' "$*"; fails=$((fails + 1)); }
info()    { printf 'INFO  %s\n' "$*"; }
section() { printf '\n== %s\n' "$*"; }
first()   { [ -r "$1" ] && head -n 1 "$1" 2>/dev/null; }

section "System"
model=$(tr -d '\000' 2>/dev/null < /proc/device-tree/model)
info "model:   ${model:-unknown}"
info "kernel:  $(uname -r) ($(uname -m))"
info "cmdline: $(cat /proc/cmdline)"
info "CPUs:    $(grep -c '^processor' /proc/cpuinfo), RAM: $(awk
'/^MemTotal/ {print int($2 / 1024) " MiB"}' /proc/meminfo)"
info "uptime:  $(cut -d' ' -f1 /proc/uptime) s"

section "Storage"
rootline=$(awk '$2 == "/" && $1 != "rootfs" {print $1, $3, $4}' /proc/mounts | tail -n 1)
case "$rootline" in
*" rw,"*|*" rw") ok "rootfs: $rootline" ;;
"")              fail "rootfs not found in /proc/mounts" ;;
*)               fail "rootfs mounted read-only: $rootline" ;;
esac
if grep -q ' /boot ' /proc/mounts; then
    ok "/boot: $(awk '$2 == "/boot" {print $1, $3}' /proc/mounts)"
    [ -r /boot/extlinux/extlinux.conf ] && ok "/boot/extlinux/extlinux.conf readable" ||
        fail "/boot/extlinux/extlinux.conf missing"
else
    fail "/boot not mounted (fstab: LABEL=BOOT)"
fi
if dd if=/dev/zero of=/root/.board-check bs=1024 count=4096 2>/dev/null && sync &&
   [ "$(wc -c < /root/.board-check)" -eq 4194304 ]; then
    ok "rootfs write test (4 MiB)"
else
    fail "rootfs write test"
fi
rm -f /root/.board-check 2>/dev/null
df -h / 2>/dev/null | tail -n 1 | sed 's/^/INFO  /'
grep -q ' /boot ' /proc/mounts && df -h /boot | tail -n 1 | sed 's/^/INFO  /'

section "Kernel"
moddir=/lib/modules/$(uname -r)
if [ -d "$moddir" ]; then
    ok "modules for this kernel: $moddir ($(find "$moddir" -name '*.ko*' | wc -l) files)"
else
    fail "no $moddir: modules do not match the kernel (rebuild the image)"
fi
info "loaded modules: $(wc -l < /proc/modules)"
if log=$(dmesg 2>/dev/null); then
```

```
    if echo "$log" | grep -q -E 'Oops|BUG:|Call trace:|Kernel panic'; then
        fail "kernel oops/BUG in dmesg: dmesg | grep -n -E 'Oops|BUG:|Call trace'"
    else
        ok "no oops/BUG in dmesg"
    fi
    n=$(echo "$log" | grep -c -i -E 'error|failed')
    [ "$n" -eq 0 ] && ok "no error/failed lines in dmesg" ||
        warn "$n error/failed lines in dmesg: dmesg | grep -i -E 'error|failed'"
else
    warn "dmesg not readable (run as root)"
fi
mount -t debugfs none /sys/kernel/debug 2>/dev/null
if [ -r /sys/kernel/debug/devices_deferred ]; then
    d=$(cat /sys/kernel/debug/devices_deferred)
    [ -z "$d" ] && ok "no deferred devices" ||
        warn "deferred devices (driver or supplier missing): $(echo "$d" | tr '\n' ';')"
fi

section "Consoles and display"
ttys=""
for p in /proc/[0-9]*; do
    cmd=$(tr '\000' ' ' 2>/dev/null < "$p/cmdline")
    case "$cmd" in
    */getty\ *|getty\ *) ttys="$ttys $(echo "$cmd" | awk '{for (i = 2; i <= NF; i++) if ($i
~ /^tty/) {print $i; exit}}')" ;;
    esac
done
[ -n "$ttys" ] && ok "login (getty) on:$ttys" || warn "no getty running"
if [ -e /dev/fb0 ]; then
    ok "framebuffer fb0: $(first /sys/class/graphics/fb0/virtual_size | tr ',' 'x'), $
(first /sys/class/graphics/fb0/bits_per_pixel) bpp, $(first /sys/class/graphics/fb0/name)"
else
    warn "no /dev/fb0 (no display driver or no display connected)"
fi
[ -r /sys/class/graphics/fbcon/rotate ] && info "fbcon rotate: $(first /sys/class/graphics/
fbcon/rotate)"
for b in /sys/class/backlight/*; do
    [ -d "$b" ] && info "backlight $(basename "$b"): $(first "$b/brightness")/$(first "$b/
max_brightness")"
done

section "Input"
found=0
for e in /sys/class/input/event*; do
    [ -d "$e" ] || continue
    found=1
    info "$(basename "$e"): $(first "$e/device/name")"
done
[ "$found" -eq 1 ] || warn "no input devices"

section "LEDs"
for l in /sys/class/leds/*; do
    [ -d "$l" ] && info "$(basename "$l"): trigger $(sed -n 's/.*\[\(.*\)\].*/\1/p' "$l/
trigger")"
done

section "Sound"
if [ -r /proc/asound/cards ] && grep -q '^ *[0-9]' /proc/asound/cards; then
    grep '^ *[0-9]' /proc/asound/cards | sed 's/^ */OK    sound card /'
else
    warn "no sound cards"
fi

section "Network"
for n in /sys/class/net/*; do
    i=$(basename "$n")
    [ "$i" = lo ] && continue
    addr=$(ip -4 addr show dev "$i" 2>/dev/null | awk '/inet / {print $2}')
    state=$(first "$n/operstate")
    if [ -n "$addr" ]; then ok "$i: $state, $addr"; else warn "$i: $state, no IPv4 address";
fi
done
```

```
[ -n "$(ip route 2>/dev/null | grep '^default')" ] && ok "default route: $(ip route | grep '^
default')" ||
    info "no default route"
pidof dropbear >/dev/null && ok "SSH server (dropbear) running" || info "no SSH server"

section "Buses"
info "GPIO chips: $(ls /dev/gpiochip* 2>/dev/null | tr '\n' ' ')"
for g in /sys/bus/gpio/devices/gpiochip*; do
    # the parent device names the controller: ff040000.gpio, 4804c000.gpio, ...
    [ -e "$g" ] && info "  $(basename "$g"): $(basename "$(dirname "$(readlink -f "$g")")")"
done
for b in /sys/bus/i2c/devices/i2c-*; do
    [ -d "$b" ] && info "I2C $(basename "$b"): $(first "$b/name")"
done
usb=$(ls /sys/bus/usb/devices 2>/dev/null | grep -c -v ':')
info "USB devices (incl. root hubs): $usb"
for u in /sys/bus/usb/devices/*; do
    [ -r "$u/product" ] && info "  $(basename "$u"): $(first "$u/product")"
done

section "Clock, sensors, power"
if [ -e /dev/rtc0 ]; then
    ok "RTC: $(first /sys/class/rtc/rtc0/name), $(first /sys/class/rtc/rtc0/date) $(first /
sys/class/rtc/rtc0/time)"
else
    warn "no /dev/rtc0"
fi
info "system time: $(date)"
for t in /sys/class/thermal/thermal_zone*; do
    [ -r "$t/temp" ] && info "$(first "$t/type"): $(( $(first "$t/temp") / 1000 )) C"
done
c=/sys/devices/system/cpu/cpu0/cpufreq
if [ -d "$c" ]; then
    ok "cpufreq: $(( $(first $c/scaling_cur_freq) / 1000 )) MHz, governor $(first $c/
scaling_governor), available: $(first $c/scaling_available_frequencies)"
else
    info "no cpufreq (CPU frequency is fixed)"
fi
for p in /sys/class/power_supply/*; do
    [ -d "$p" ] && info "power $(basename "$p"): $(first "$p/type") $(first "$p/status") $(fi
rst "$p/capacity")"
done

section "Summary"
[ "$fails" -eq 0 ] && echo "no FAIL lines" || echo "$fails FAIL line(s)"
exit "$fails"
```